Dr. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE

SHRI TULJABHAVANI COLLEGE OF ENGINEERING, TULJAPUR.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(2024-25)

SIGN LANGUAGE TRANSLATOR

Submitted by

Name: Prathamesh N. Raut **Roll No.:** 3543 **PRN No.:** 2221311242055

Name: Yogesh B. Reddy **Roll No.:** 3544 **PRN No.:** 2221311242056

Name: Aniket S. Tandale **Roll No.:** 3533 **PRN No.:** 2221311242042

Under the guidance of

Prof. S. E. Hajare

DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE

CERTIFICATE

This is to certify that the Mini Project report entitled "**Sign language Translator",** submitted by Mr**. Prathamesh Raut, Mr. Yogesh Reddy, Mr. Aniket Tandale** is the bonafide work completed under my guidance in the partial fulfillment for the award of Third Year (TY) in Computer Science and Engineering of Dr. Babasaheb Ambedkar Technological University, Lonere.

Place: Tuljapur

Date:_____

Prof. S. E. Hajare
Guide Name

Prof. J. M. Shaikh
HOD
Department of CSE

Dr. Omprakash Rajankar
Principal
STB College of Engineering, Tuljapur

ACKNOWLEDGEMENT

With immense pleasure We, Prathamesh, Yogesh, Aniket presenting "**Sign Language Translator**" Mini Project report as a part of the curriculum of Third Year (TY) in Computer Science & Engineering. I wish to thank all the people who gave me unending support. I take this opportunity to express my deep sense of gratitude to my guide Prof. S. E. Hajare, for their valuable guidance and inspiration and also to our Principal Dr. Omprakash Rajankar Sir for their encouragement and notable guidance. Our special thanks to Prof. J. M. Shaikh, HOD of CSE department and staff who gives precious guidance.

Date: _____

Abstract

Sign Languages (SLs) are employed by deaf and hard-of-hearing (DHH) people to communicate on a daily basis. However, the communication with hearing people still faces some barriers, mainly because of the scarce knowledge about SLs among hearing people. Hence, tools to allow the communication between users of either sign or spoken languages must be encouraged. A stepping stone in this direction is the research of the sign language translation (SLT) task, which aims to produce a spoken language translation of a sign language video or vice versa. By implementing these types of translators in portable devices, we will make considerable progress towards a barrier-free communication between DHH and hearing people. That is why, in this work, we focus on reviewing the literature on SLT and provide the necessary background about SLs. Besides, we summarise the available datasets and the results found in the literature for one of the most used datasets, the RWTH-PHOENIX-2014T. Moreover, the survey lists the challenges that need to be tackled within the SLT research and also for the adoption of SLT technologies, and proposes future research lines.

TABLE OF CONTENTS

1. INTRODUCTION

Communication is the cornerstone of human interaction and social cohesion. For most of us, speech and hearing provide a seamless channel to express thoughts and emotions. However, for millions in the deaf and mute community, these fundamental interactions rely on a distinct medium—sign language. Indian Sign Language (ISL) is widely used in India, yet it is not understood by the vast majority of the population, creating an invisible barrier that restricts the full integration of the hearing-impaired into society.

Despite technological advancements in communication, a significant gap still exists when it comes to supporting non-verbal communication in a multilingual and diverse country like India. While American Sign Language (ASL) has received some attention in the research and development community, Indian Sign Language remains vastly underrepresented in accessible technologies. Most sign language translators focus solely on English or do not include real-time capabilities, user-friendly designs, or support for Indian regional languages.

To address this gap, we propose **"SignSpeak"**—a real-time AI-based Indian Sign Language translator that not only recognizes gestures but also converts them into audible speech in a selected native Indian language. This project is an amalgamation of computer vision, deep learning, natural language processing (NLP), and multilingual user interface design. By incorporating speech synthesis in regional languages, our project provides inclusive communication solutions that extend beyond literacy and language proficiency.

The core idea behind SignSpeak is to make communication easier for those who use sign language by offering a system that can translate signs into understandable speech for the listener, and vice versa, without needing the user to be literate or English-speaking. This is especially crucial in rural areas, where access to education and English language resources can be limited.

The project's motivation also lies in the need for greater inclusion in everyday interactions, whether it's a doctor understanding a patient's concern, a government officer helping with paperwork, or a family member understanding a deaf relative's needs. Technology should empower individuals, and SignSpeak aims to do just that—making communication possible where it was previously difficult or non-existent.

Technically, this project combines the robustness of a convolutional neural network (CNN) to classify hand gestures with the practicality of a React.js front-end interface and a Python-Flask backend. The application supports both real-time webcam-based input and static image uploads,

provides translation in multiple Indian languages, and includes voice output for a complete end-to-end communication system. The UI is kept minimal and icon-based to help users with little or no formal education.

Moreover, SignSpeak is not just a software solution—it is a statement of accessibility, inclusivity, and the power of AI to make a social difference. It represents a step toward a more inclusive digital ecosystem where language and literacy are not barriers to communication. With an emphasis on community, accessibility, and real-time interaction, SignSpeak redefines what assistive technology can achieve in the Indian context.

By bridging the gap between gesture and spoken language, SignSpeak fosters understanding, breaks down barriers, and promotes empathy across communication divides. Through its thoughtful design and technological innovation, this project contributes meaningfully to the vision of digital inclusion and accessibility for all.

2. LITERATURE SURVEY

Several studies and projects have addressed gesture recognition, especially using ASL (American Sign Language), which has seen considerable attention from researchers worldwide. However, most of these systems are targeted at English-speaking users and involve sophisticated hardware or complex implementations.

- **Malchanov et al.** utilized a skin tone detection method in the YCbCr color space for ASL classification. Their approach involved adaptive filtering based on ethnicity-specific tone ranges, improving hand region segmentation in varying light conditions. This study helped reinforce the importance of accurate preprocessing in hand gesture recognition.
- **Pigou et al.** proposed a technique using 3D Haar-like features and depth sensors like Kinect to recognize static hand gestures. Their model focused on spatial and depth-based pattern recognition using deep belief networks. While their accuracy was impressive, it required expensive hardware that may not be feasible in low-income settings.
- **Nasri et al.** developed a mobile-based gesture recognition system using gesture histograms and Binary Robust Independent Elementary Features (BRIEF). Their app aimed at making sign-to-text translation accessible but still lacked multilingual support and could not form grammatically correct sentences from sequences of signs.
- **Huang et al.** emphasized motion-based recognition, including facial expressions and upper-body posture, to capture more nuanced and context-rich gesture inputs. Their work showcased the effectiveness of incorporating multimodal data but introduced a significant increase in system complexity.

While these works demonstrate promising approaches to gesture recognition, there are common limitations:

- **Language Limitation:** Most systems focus solely on English, making them unsuitable for India's diverse linguistic landscape.
- **Cost and Hardware Dependency:** Many rely on depth sensors or high-end computing resources, limiting accessibility in rural areas.
- **Usability:** Most systems assume a literate, technologically adept user and often overlook inclusivity and intuitive UI/UX.

- **Lack of Real-Time Sentence Structuring:** Few systems can interpret gestures as parts of full sentences or form meaningful phrase outputs.

SignSpeak's Contribution: Unlike these prior works, SignSpeak is built for accessibility and local relevance. It uses a lightweight Convolutional Neural Network (CNN) that can operate on webcam inputs, removing the need for specialized cameras or hardware. It also features a native language UI using i18next, integrates audio feedback for illiterate users, and is optimized for browsers—requiring no complex installation. Furthermore, SignSpeak includes a sentence formation system, history tracking, and real-time speech synthesis in Indian languages, which makes it a more holistic, inclusive, and scalable solution for sign language translation in the Indian context.

3. OBJECTIVES

The objective of our project is to bridge the communication gap between the hearing-impaired community and the rest of society through an accessible, real-time sign language translation tool. The following objectives guide the development and implementation of SignSpeak:

• To design a deep learning model capable of recognizing and classifying Indian Sign Language gestures with high accuracy.

• To integrate real-time prediction via webcam and static image upload to allow flexible input modes.

• To provide speech output in selected native Indian languages for inclusivity among illiterate users.

• To develop an intuitive, icon-based user interface suitable for non-literate and rural populations.

• To implement a sentence-building feature that compiles a sequence of signs into coherent phrases.

• To maintain a prediction history for review or sharing.

• To ensure cross-browser compatibility and responsive design.

• To facilitate easy integration with external platforms (WhatsApp, government services, etc.) in future iterations.

In addition to the core objectives, our system is built to support real-world use cases such as healthcare communication, educational assistance, and public service accessibility. SignSpeak is designed to function as a scalable solution that can be integrated into digital kiosks, mobile applications, and desktop environments.

Another goal is to bridge the gap between artificial intelligence and accessibility. We strive to minimize the digital divide that leaves people with disabilities at a disadvantage in the age of smart technologies. By offering a real-time system that is compatible with low-cost devices, SignSpeak aims to provide functionality even in areas with limited infrastructure.

We also seek to ensure that the system is inclusive not only from a language standpoint but also from a usability perspective. The design accommodates users who may be visually impaired or have other motor difficulties by including voice feedback, large on-screen icons, and minimal text-based interaction.

Moreover, the system is built with modularity in mind. The prediction model, UI, and audio generation layers can be independently updated or replaced in the future, ensuring long-term adaptability. This modularity allows us to scale the system for additional sign languages, integrate gesture-to-speech in conversational formats, and even introduce AI-driven grammar correction.

SignSpeak's roadmap includes extending its use to government portals, helping individuals communicate when applying for documents, services, or benefits. We also envision this project becoming a foundational framework for future research in accessible communication systems.

Ultimately, our objective is not only to build a tool but to initiate a movement toward inclusivity in everyday technology.

4. PROBLEM STATEMENT

Despite the rapid advancement of artificial intelligence and machine learning in recent years, there remains a stark gap in inclusive communication technologies for the hearing- and speech-impaired communities in India. A majority of existing sign language translation tools focus on American Sign Language (ASL), often ignoring the specific gestures, cultural nuances, and language diversity found in Indian Sign Language (ISL).

Even the few ISL recognition systems that do exist are typically designed with assumptions about the user—namely that they are literate, tech-savvy, and proficient in English. This design limitation poses a major challenge in India, where a significant portion of the population either does not speak English or has limited access to digital literacy and educational resources. These systems often lack native language support, audio feedback, and accessible interfaces, making them impractical for real-world deployment in rural or underserved areas.

Furthermore, most current systems provide recognition only at the individual sign level, without the ability to form sentences or contextual meaning. This limits their usability in real-time conversation, which often requires more than identifying a single word or alphabet.

Given these limitations, our project addresses the following core problems:

- Lack of support for Indian Sign Language (ISL) in mainstream assistive technologies.
- Absence of native Indian language support and audio output for illiterate users.
- Inaccessible and complex user interfaces unsuitable for rural or non-literate users.
- Inability to translate signs into complete, coherent sentences.
- Limited real-time performance due to inadequate webcam support or model efficiency.

SignSpeak aims to solve these problems through the integration of real-time gesture recognition, sentence formation, audio feedback, and a multilingual user interface that is inclusive, simple, and scalable.

5. EXISTING SYSTEM VS PROPOSED SYSTEM

Feature	Existing Systems	SignSpeak
Native Language Support	No	Yes
Audio Output	Limited	Yes (Speech Synthesis)
UI for Non-Literate Users	No	Yes
Webcam Support	Some	Yes
History Management	Rare	Yes
Real-time Sentence Formation	No	Yes

The existing systems tend to focus only on recognizing ASL and are generally not adaptable to Indian linguistic and cultural requirements. The proposed system addresses these limitations with a tailored solution for the Indian demographic.

6. SYSTEM REQUIREMENTS

To ensure optimal performance, responsiveness, and compatibility across different user environments, the system requirements for developing and deploying SignSpeak are divided into hardware and software categories. These specifications are chosen keeping in mind the balance between computational efficiency and widespread accessibility, especially for institutions and users with limited resources.

Hardware Requirements:

- Processor: A minimum of Intel Core i5 (or AMD equivalent) is required to run the application smoothly, especially during real-time inference.
- Memory (RAM): At least 8 GB RAM is recommended to handle live webcam feeds, run the model, and support browser-based interactions simultaneously.
- Camera: A built-in or external webcam is necessary to capture live hand gestures for real-time prediction.

- Graphics Processing Unit (GPU): Although optional for end-users, a GPU (such as NVIDIA GTX 1050 or better) is highly recommended during the training phase of the deep learning model to speed up epochs and improve performance. For deployment, GPU usage is optional as the trained model is optimized for CPU inference.

Software Requirements:

- **Operating System:** The application is cross-platform and compatible with Windows 10+, Ubuntu 20.04+, and other modern Linux distributions.
- Programming Languages & Runtimes:
    - Python 3.11 or newer (for model inference, backend logic, and OpenCV processing).
- Node.js 18+ (for React-based frontend execution).
- **Frameworks and Libraries:**
    - React.js (frontend framework for building UI components).
    - Flask (lightweight backend API to connect the model with the frontend).
    - TensorFlow/Keras, OpenCV, pyttsx3, i18next, and supporting packages.

- **Web Browsers:** The application supports and has been tested on the latest versions of Chrome, Firefox, and Microsoft Edge. Browser support ensures easy access without requiring installation.
- **Development Environment:** Visual Studio Code (VS Code) is recommended due to its flexibility, extensions for Python/JavaScript, and integrated terminal for running both frontend and backend servers.

These specifications ensure the system functions smoothly across common educational or institutional settings, especially during real-time webcam-based gesture recognition and native speech output. The software stack also allows for scalable updates, cross-device portability, and cloud integration if needed in future deployments.

7. TOOLS AND TECHNOLOGIES USED

This section elaborates on the tools, programming languages, and frameworks utilized in the development of SignSpeak. The technological stack is divided into frontend, backend, and machine learning components for clarity.

Frontend:

- **React.js:**
  - A powerful JavaScript library used to build a responsive and dynamic UI.
- **Bootstrap:**
  - For building responsive layouts and components.
- **HTML5/CSS3/JavaScript:**
  - Core web technologies for structure, styling, and interactivity.
- **i18next / React-i18next:**
  - For implementing multilingual interfaces.
- **React Webcam:**
  - Enables capturing frames directly from the webcam.

Backend:

- **Flask:**
  - A Python micro web framework used to create RESTful APIs.
- **REST API:**
  - Used for communication between frontend and backend.
- **Pyttsx3 / gTTS / SpeechSynthesis API:**
  - Used for converting text to speech output.

Machine Learning & Libraries:

- **TensorFlow/Keras:**
  - Used for training and deploying the CNN model for sign recognition.
- **OpenCV:**
  - Handles image processing like resizing, color conversion, and filtering.
- **NumPy/Pandas/Matplotlib:**
  - Used during model training and evaluation.

8. SYSTEM ARCHITECTURE

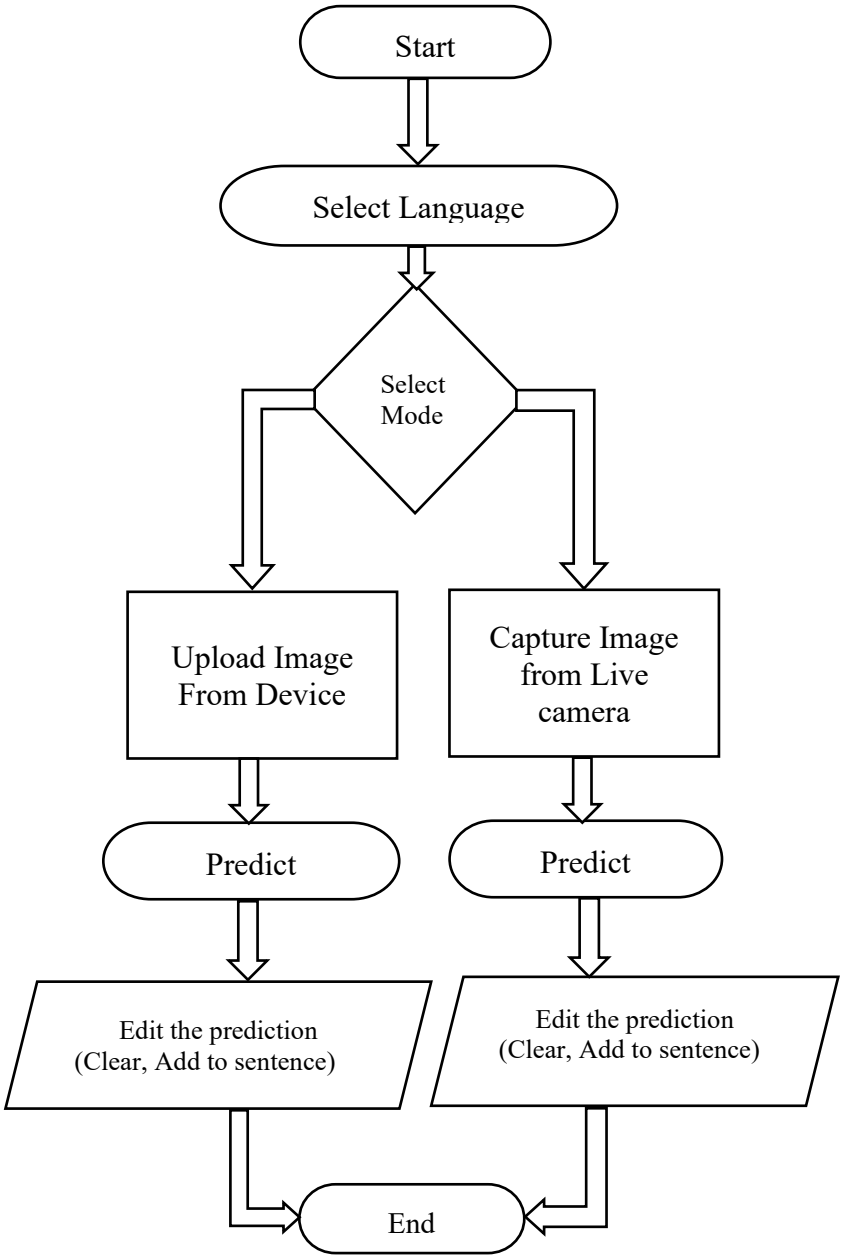
The architecture of SignSpeak follows a modular and layered design to separate concerns and ensure maintainability.

1. **User Layer:**
    - Includes the web-based user interface created with React.js. Users interact with the application through this layer.
2. **Application Layer:**
    - Handles logic, API requests, and user interactions. Implemented using Flask for routing and middleware.
3. **Model Layer:**
    - Processes image input through a pre-trained CNN model to classify hand gestures.
4. **Output Layer:**
    - Converts predictions into text and optionally speech using audio synthesis modules.

Data Flow Overview:

- Webcam or image input is captured and sent to the Flask backend via HTTP POST.
- The backend pre-processes the image using OpenCV.
- The trained CNN model predicts the gesture.
- The result is sent back to the frontend.
- Text is shown, sentence is formed, and optionally converted to audio output

❖ **Data-Flow Diagram**



9. MODULE DESCRIPTION

The SignSpeak application is built with a modular architecture, where each module handles a specific functionality of the system. This approach ensures clarity, scalability, ease of testing, and future enhancement. Below are the core modules involved in the system:

- **Language Selector Module:**
    - This module provides users with an option to choose their preferred language from a list that includes Hindi, Marathi, Bengali, and other Indian languages.
    - It is built using React hooks and i18next to dynamically render UI components in the selected language.
    - The chosen language also determines the voice output used in the Text-to-Speech module, enhancing localization.
- **Image Upload Module:**
    - Allows users to upload an image containing a hand gesture for prediction.
    - The uploaded image is processed in the backend using OpenCV to standardize dimensions and contrast before feeding it to the model.
    - This module is useful in educational or offline scenarios where static gestures are captured and then interpreted.
- **Webcam Prediction Module:**
    - Captures real-time frames from the webcam using the React Webcam package.
    - The captured image is converted into base64 format and sent via API to the backend.
    - This module supports continuous interaction, where users can predict multiple signs without needing to upload each manually.
- **Sentence Formation Module:**
    - Once a hand gesture is recognized, this module adds the corresponding alphabet or word to a text buffer to form meaningful sentences.
    - The sentence can be edited using additional buttons like Backspace, Clear, or Submit.
    - It stores the intermediate outputs, allowing users to construct longer conversations.
- **Text-to-Speech Module:**

- Converts the predicted sentence into audio output in the user's selected language.
- The module uses either pyttsx3 (for Python desktop apps) or the Web Speech API (for browsers) for synthesizing speech.
- This module is particularly valuable for enabling verbal communication with those who cannot read text.

- **History Log Module:**
  - Maintains a log of past predictions for user review.
  - Each prediction includes timestamp, image source (upload/webcam), recognized text, and selected language.
  - Users can download or export this history for educational or documentation purposes.
- **Control Panel Module:**
  - Provides buttons for performing essential operations: Clear (reset buffer), Backspace (delete last entry), Speak (read sentence aloud), Submit (store history), and Download (save sentence).
  - Designed with accessibility in mind—large buttons, icon labels, and responsive layout.

Each module operates in coordination with others to provide a seamless and inclusive user experience. The modular design also allows for independent upgrades, such as adding more languages, introducing AI-based grammar correction in sentence formation, or supporting gesture video clips in the future.

10. TESTING AND VALIDATION

Thorough testing and validation procedures were carried out to ensure the reliability, usability, and performance of the SignSpeak system across devices and scenarios:

1. **Unit Testing:**
    - Each module (Webcam input, Upload form, Language switch, etc.) was tested in isolation.
    - JavaScript unit testing tools like Jest were used for front-end components.
2. **Model Validation:**
    - The CNN model was trained on the asl_alphabet_train dataset.
    - A validation split of 20% was used to evaluate accuracy and detect overfitting.
    - Achieved over 96% validation accuracy after data augmentation and 20 epochs of training.
3. **UI/UX Testing:**
    - Tested across Chrome, Firefox, and Edge browsers.
    - Responsive design verified across desktops, tablets, and phones.
    - Feedback from users with minimal technical experience was collected to improve navigation.
4. **Cross-Language Verification:**
    - The i18next implementation was validated using Hindi, Marathi, and English.
    - Voice outputs were reviewed for pronunciation accuracy.
5. **Real-World Testing:**
    - Prediction performance was tested in varying light conditions using a standard webcam.
    - Performance bottlenecks were logged and reduced by optimizing image conversion and resizing.

11. CHALLENGES FACED

While developing SignSpeak, we encountered multiple technical and conceptual challenges, including:

1. **Real-Time Performance:**
    - Managing webcam input, base64 encoding, and API latency without noticeable delay required optimization of backend prediction speed and frontend UI refresh cycles.
2. **Accuracy vs. Speed Tradeoff:**
    - Higher accuracy often came at the cost of inference speed. A balance was achieved using lightweight CNN architecture and input image resizing.
3. **Gesture Ambiguity:**
    - Some signs resemble each other closely (e.g., M and N in fingerspelling), leading to misclassification.
    - Training data was augmented and noise-robust preprocessing was added.
4. **Localization:**
    - Ensuring seamless switching between native languages without crashing the UI required careful implementation using language hooks in React.
5. **Cross-Device Compatibility:**
    - Users on mobile browsers sometimes encountered permission issues with camera and microphone access.
    - Fallbacks and clear permissions guidance were added.
6. **User Accessibility:**
    - Designing an interface that is intuitive even for users who are illiterate or unfamiliar with computers posed a unique challenge.
    - We prioritized large icons, fewer text elements, and visual cues wherever possible.

12. CONCLUSION

SignSpeak is more than a sign language translation tool—it is an inclusive communication bridge that integrates real-time AI, language localization, and a user-first design. This system enhances accessibility and empowers the deaf and mute community to communicate effectively with others who may not know Indian Sign Language (ISL). By leveraging machine learning, natural language processing, and modern frontend-backend integration, we created a platform that not only translates signs but speaks them aloud in native languages. The real-world applicability, especially in rural and non-English speaking regions, opens a path toward a more inclusive India.

This project also showcases how technology, when thoughtfully applied, can significantly impact marginalized communities. The implementation of a multilingual UI, voice synthesis, and simple, icon-driven interactions ensures that users of varying literacy levels can use the application with ease. While challenges in real-time prediction, hardware limitations, and language models were faced, SignSpeak's current implementation successfully addresses the key gaps found in existing solutions.

13. REFERENCES

- ✓ Kaggle. “ASL Alphabet Dataset.” https://www.kaggle.com/datasets/grassknoted/asl-alphabet
- ✓ TensorFlow Documentation. https://www.tensorflow.org/
- ✓ Flask Documentation. https://flask.palletsprojects.com/
- ✓ React Documentation. https://reactjs.org/
- ✓ MDN Web Docs. https://developer.mozilla.org/
- ✓ React Webcam Library. https://www.npmjs.com/package/react-webcam
- ✓ pyttsx3 Text-to-Speech. https://pypi.org/project/pyttsx3/
- ✓ OpenCV Documentation. https://docs.opencv.org/
- ✓ i18next Documentation. https://www.i18next.com/